KU LEUVEN kulak

Faculty of Science & Technology Kulak

Introduction to 2D physics engines

Constraint-based rigid-body simulations

Casper Vermeeren

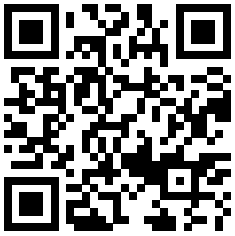
Physics

Academic year 2025 – 2026

Introduction to 2D physics engines

Casper Vermeeren

Academic year 2025 – 2026



Scan this QR code to access the accompanying video material for this book. The website contains video simulations along with energy plots, providing critical insight into actually applying the derived concepts.

Contents

Introduction

WIP

Chapter 1

Unconstrained dynamics

1.1 The basics

1.1.1 The state vector

UPDATE THIS WITH X AND V SEPERATELY (WIP)

The state vector is a tool used to describe one time-frame of the entire system. Given this vector, one should be able to perfectly reproduce an image of the current situation. From this description it thus also follows that the state vector and all of its components are - or can be - time-dependent. A two-dimensional system describing two particles would have a state vector with eight components: 2 parts (position + velocity) * 2 dimensions * 2 particles. A general 2D system with N particles has a state vector of length 4N.

$$\vec{X}(t) = \begin{pmatrix} x_1(t) \\ y_1(t) \\ x_2(t) \\ y_2(t) \\ v_{x1}(t) \\ v_{y1}(t) \\ v_{x2}(t) \\ v_{y2}(t) \end{pmatrix}$$

The goal of solving and simulating such a physical system is to describe the derivative of this state vector. It's very clear that the derivatives of the first four components here are simply the last four components, but the derivatives of the velocities are a tad more complex. Moreover it depends on which situation we are looking at.

1.1.2 Forces and their components

Velocities change through acceleration, caused by forces. If we can calculate a force using some equation, it can easily be converted to an acceleration via $F = ma$. This acceleration, however, is still a vector. One must therefore be able to dynamically split the acceleration vector into the components a_x and a_y, such that they can be added as parts of the velocity derivative. This also requires the perpendicular x- and y-axes to be defined in advance.

We will generally cover two types of forces: external forces and internal forces. While not a one-to-one relation, you could also differentiate these as known forces and constraint forces. The former are forces we can calculate using a known equation, such as gravity ($F = mg$), spring force ($F = kx$) or air resistance ($F = -bv$). The latter are harder to calculate, since they will only be known once the system has been entirely solved. Examples are the normal force, the tension force and any other type of force that keeps objects fixed or in some shape. These constraint-dependent forces will be derived in more complex ways and will also be the main reason why dynamic simulations are not exact - a system whose energy should theoretically be conserved might have slight deviations because of these approximations.

1.2 Why not Lagrangian mechanics?

With these constraint-dependent caveats in mind, you could suggest using Lagrangian mechanics for our simulations. This is a great recommendation on the surface, as using the Lagrangian allows us to convert (holonomic) constraints into reductions in coordinates and a great way to derive the differential equations of motion. Furthermore, whatever Lagrangian mechanics outputs, will - under a proper integrator - deliver physically exact solutions.

The only problem is that this method is extremely situation-dependent. Major steps such as the choice of generalized coordinates and the expressions of kinetic and potential energy are hard to automate in code. If we had a magic box that we could throw a problem into and it spit out the necessary coordinates and accompanying energy expressions, then technically the remaining steps could be automated without issue. It just needs to compute the Lagrangian $T - V$ and solve a few partial derivatives to find the equations of motion, which can then be simulated with a numerical integrator.

Even with that, simulators using Lagrangian mechanics do of course exist. The downside of them is their lack of complete freedom in what you wish to create. Certain typical constraints like the double pendulum are often hard-coded, same for potentials like gravity. What we want is a fully dynamic system that can simulate anything you can dream of - the only limit being numerical approximation and your computer's price tag.

1.3 Numerical time integrators

1.3.1 Runge-Kutta

1.3.2 Verlet

1.4 Known or external forces

1.4.1 Gravity

1.4.2 Air resistance

1.4.3 Springs

If two particles are connected by a spring, then the spring force on one of the two particles is given as

$$\vec{F} = k(l - l_0)\Delta\vec{r}$$

where $\Delta\vec{r}$ is the unit vector pointing from that particle (to whom the force applies) to the other particle. In the case of two particles, the length l can be calculated as

$$l = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

which corresponds to the length of the vector connecting the two particles. This unit vector can thus be found as

$$\Delta\vec{r} = \begin{pmatrix} \frac{x_2 - x_1}{l} & \frac{y_2 - y_1}{l} \end{pmatrix}$$

for the first particle and opposite for the second particle - which could also be derived from Newton's third law. In total, we thus find that

$$F_{1x} = k(l - l_0)\frac{x_2 - x_1}{l}$$

$$F_{1y} = k(l - l_0)\frac{y_2 - y_1}{l}$$

$$F_{2x} = k(l - l_0)\frac{x_1 - x_2}{l}$$

$$F_{2y} = k(l - l_0)\frac{y_1 - y_2}{l}.$$

Dividing by m_1 or m_2 to find components of acceleration gives us the full state vector for this setup, which can then be simulated.

Chapter 2

Principles of constraints

The second category of forces to model are the constraint or internal forces. As mentioned before, these are forces that arise from constraints or limits on how our system can move around. Examples include

- pendulums, where two points are connected by a rod of fixed length, where this fixing is the constraint.
- walls or boundaries, where particle positions are limited to a specific interval. These constraints are non-holonomic, meaning they are expressed as inequalities and not as equalities, which poses an even tougher challenge.
- fixed points, where an object (like a rod) is fixed at some point and can only rotate around this point.

The challenge in modeling these forces is that, unlike known or external forces, there are no textbook-defined equations for any of them. They depend heavily on the system itself and most notably on the motion it will perform. This means that to know the forces, we technically need to solve the motion. But to solve the motion, we also need the forces! In other words, the constraints themselves define the forces. This is not ground-breaking: even in basic Newtonian physics, things like normal force can be found by imposing that an object is at rest vertically and will thus need this upward force to equal downward gravity in magnitude.

To model forces that originate from constraints on our system, we'll need some very important mathematical toolkit. This will involve virtual displacements and virtual work, as well as how constraints should affect the balance of forces mathematically, and finally how to solve the implied optimization problems. In the third chapter we'll then generalize this for any amount of constraints using linear algebra.

2.1 Virtual displacements and virtual work

First, let's properly define what these 'virtual' shenanigans are.

Definition 2.1.1. A virtual displacement $\delta\vec{r}$ is an infinitesimal, time-independent, hypothetical displacement that respects all constraints of the system and has no relation to real displacements.[6]

At this point you should know what an infinitesimal is and how you can symbolically derive it. For example, in Lagrangian mechanics we say positions $\vec{r}_i$ depend on generalized coordinates $q_1, q_2, \ldots, q_n$ as well as time t, such that

$$d\vec{r}_i = \sum_{j=1}^{n} \frac{\partial \vec{r}_i}{\partial q_j} dq_j + \frac{\partial \vec{r}_i}{\partial t} dt.$$

The difference for a virtual displacement $\delta\vec{r}_i$ is that it happens in an instant; we do not take time dependency into account. Therefore we also neglect the actual time derivative, leaving us with

$$\delta\vec{r}_i = \sum_{j=1}^{n} \frac{\partial \vec{r}_i}{\partial q_j} \delta q_j.$$

Note that the change in the generalized coordinates has also been turned into a virtual change. Next let's introduce virtual work, as it's a simple extension.

Definition 2.1.2. Virtual work is the work done by some force over a virtual displacement.[6]

$$\delta W = \vec{F} \cdot \delta\vec{r}$$

In statics analysis - like for engineering applications - virtual work can be used to more efficiently find the size of internal forces. This is because in statics the total force is zero, such that the total virtual work for any allowed virtual displacement is also zero. Smart choices for the virtual displacement can then lead to some forces disappearing due to perpendicularity in the dot product, leaving relations between unknown forces and known forces.

We however are dealing with dynamic systems, they're not static at all. This sadly means the total force is no longer zero, but due to the way virtual displacements are defined, they will very often lead to ideal constraint forces.

Definition 2.1.3. For some dynamic system, a constraint force is ideal if the virtual work performed by it over an allowed virtual displacement is zero.[6]

$$\vec{F}_{ideal} \cdot \delta\vec{r} = 0$$

This arises when your constraint force is perpendicular to the virtual displacements your system allows. And while it may sound rare or incredibly specific, it happens way more frequently than you'd think.

- Imagine a hockey puck sliding over the ground. The primary constraint in this system is that the puck is lying still on the ice - no vertical motion. This is created by a constraint force, the normal force, acting perpendicular to the ice floor. Any allowed virtual displacement, that thus respects the constraints, is simply the puck sliding horizontally over the ice. In

other words, the virtual displacement will always be tangential to the ice floor. Because the force is now always perpendicular to the virtual displacement, the virtual work here is zero and this force is an ideal constraint force.

- Imagine the single pendulum example. The pendulum's rod has a fixed length, which is exactly the constraint. To assure this, there is a constraint force acting along the rod. The only virtual displacements that are allowed are rotational movements, so the movement vectors are tangential to the circle that the rod describes. Once again force and virtual displacement are perpendicular, causing an ideal constraint force with no virtual work.

2.2 D'Alembert's principle

D'Alembert's principle in dynamics makes use of ideal constraint forces to find a relation between the known, external forces and the time derivative of momentum.

Theorem 2.2.1 (D'Alembert's Principle). For a set of point masses under the influence of external forces $\vec{K}_i$ and internal forces, where the latter are ideal, it holds for some virtual displacements $\delta\vec{r}_i$ that[6]

$$\sum_{i=1}^{n} (\vec{K}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0.$$

Proof. We start at Newton's second law. It states that the derivative of momentum for some particle is equal to all forces acting upon it. This includes both external (known) forces $\vec{K}_i$ and internal (constraint) forces $\vec{N}_i$,

$$\forall i : \vec{K}_i + \vec{N}_i = \dot{\vec{p}}_i.$$

Without loss of generality we can introduce a virtual displacement $\delta\vec{r}_i$ that respects all the system's constraints,

$$\forall i : (\vec{K}_i + \vec{N}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0$$

The keen eye may notice that this momentum derivative could be regarded as an inertial force. Now we use the theorem's assumption that all constraint forces are ideal under allowed virtual displacements. This causes an entire term to disappear. Along with that, we can take the sum and thus conclude that

$$\sum_{i=1}^{n} (\vec{K}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0.$$

□

D'Alembert's principle is the basis of Lagrangian mechanics. It can be used as one of multiple methods to derive its most important equation, namely $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_j}\right) - \frac{\partial L}{\partial q_j} = 0$. While we will not be using this approach here, what we're about to cover is completely equivalent. It results in the same physics, albeit more free, complicated and numerically solvable. Both principles can also be derived from each other.

2.3 Gauss's principle of least constraint

To understand Gauss's principle of least constraint, we must think about how constraints are actually incorporated into simulations.

The first step is always to think about known forces. Calculate those using their "famous" equations and sum their vectors to find the total external force acting upon each particle. If you just run the simulation now, no constraints are being taken into account: rods that should have a fixed length will stretch out forever, objects will fall through floors and walls will turn into thin air.

To satisfy constraints, we will obviously need to introduce the internal forces. After doing so, the actual acceleration (or total force) acting on the particle will have changed. The insight Gauss had is that technically speaking there are infinitely many possible ways to edit your acceleration in order to have constraints satisfied. Just imagine the single pendulum. The two ends of the rod - or every particle in the rod - feel internal forces that altogether satisfy the constraint, when combined with the external forces. But hypothetically you could make the internal force on one end a bit stronger, and just make the force on the other end an equal amount weaker. Same motion, different individual changes in acceleration.

So which option out of all of them should we pick to satisfy our constraints? Like with a lot of things in physics, it comes down to a variational formulation: we're going to minimize some quantity.

Theorem 2.3.1 (Gauss's Principle of Least Constraint). To correctly describe a system of particles under a given constraint, where m_i are their individual masses, $\vec{K}_i$ are the external forces, the quantity

$$Z = \sum_{i=1}^{n} m_i \left| \ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right|^2$$

must be minimized to some final accelerations $\ddot{\vec{r}}_i$ under those same constraints.[12] [7]

Proof. We will derive the principle from D'Alembert's Principle. Recall the statement

$$\sum_{i=1}^{n} (\vec{K}_i - \dot{\vec{p}}_i) \cdot \delta\vec{r}_i = 0.$$

Next we assume that the particle masses remain constant, such that we can rewrite momentum and find

$$\sum_{i=1}^{n} (\vec{K}_i - m_i \ddot{\vec{r}}_i) \cdot \delta\vec{r}_i = 0.$$

These $\ddot{\vec{r}}_i$ are the resulting accelerations, so they can be split into effects of external forces, as well as a correction - the internal forces part - to satisfy constraints,

$$\ddot{\vec{r}}_i = \frac{\vec{K}_i}{m_i} + \vec{\alpha}_i.$$

We can also rewrite D'Alembert as

$$\sum_{i=1}^{n} m_i \left(\ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right) \cdot \delta \vec{r}_i = 0$$

or equivalently with the above definition,

$$\sum_{i=1}^{n} m_i \vec{\alpha}_i \cdot \delta \vec{r}_i = 0. \tag{2.1}$$

This homogeneous equation can also be viewed from a variational perspective. The coordinates we are varying are the constraint-imposed corrections $\vec{\alpha}_i$ - also captured in the final accelerations. If we introduce

$$Z = \frac{1}{2} \sum_{i=1}^{n} m_i \left| \vec{\alpha}_i \right|^2$$

then a virtual change in Z can be written as

$$\delta Z = \sum_{i=1}^{n} m_i \vec{\alpha}_i \cdot \delta \vec{\alpha}_i.$$

Since virtual changes in constraint-induced corrections $\delta \vec{\alpha}_i$ are directly related to our virtual displacements $\delta \vec{r}_i$ - the constraints define which displacements are allowed - then from 2.1 it follows that $\delta Z = 0$. In other words, we must variationally tweak $\vec{\alpha}_i$ to find stable extreme values of Z - minimums. The $\frac{1}{2}$ term will not impact this, and we can substitute back our definition of $\vec{\alpha}_i$ to find that we need to solve for a minimum of the quantity

$$Z = \sum_{i=1}^{n} m_i \left| \ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right|^2.$$

□

As mentioned previously, this new principle is in fact completely equivalent to D'Alembert's Principle. This should not be shocking, as we just derived one from the other. The difference is in how both of them describe the same mechanics. D'Alembert's Principle describes an equivalence-based solution. Through solving it in Lagrangian mechanics, one will use differential equations to describe the motion of an entire system. What we've done for Gauss' Principle of Least Constraint is convert it into an optimization problem. Now we can simply find the final accelerations $\ddot{\vec{r}}_i$ as those that minimize the quantity Z[1]. Furthermore, if we were to subtract the external accelerations, we can find the mechanically correct internal forces.

2.4 Lagrange multiplier for constrained optimization

Now that we've converted our search for the internal forces into an optimization problem - with the final accelerations as coordinates - we just need a mathematical tool to solve it. This is not a normal optimization problem, where we're going to calculate some partial derivatives and call

[1] Z stands for the German "Zwang", literally meaning "compulsion" or "constraint". This is the nomenclature Gauss himself used.

it a day. The minimum also has to respect the system's constraints.

We can once again draw the comparison to using D'Alembert's Principle. Since this is an equation, not an optimization problem, the constraints are now encoded in your coordinates. Specifically, remember the fact that when solving a system with Lagrangian mechanics, generalized coordinates must be chosen. That's because every (holonomic) constraint present in the system rules out one coordinate. With Gauss's Principle things are different: we're still using Cartesian coordinates and the constraints are simply extra requirements on top of the fact our force/acceleration result must minimize some quantity.

So how does one solve a constrained optimization problem? In other words, given a function $f(\vec{x})$, where $\vec{x}$ are all the coordinates, and a (holonomic) constraint on those coordinates, given by some $g(\vec{x}) = 0$, how do you find the $\vec{x}^*$ that both minimizes f and respects g?

Theorem 2.4.1 (Lagrange Multiplier Theorem). To solve a constrained optimization problem, where $f(\vec{x})$ is the function to minimize/maximize, $g(\vec{x}) = 0$ is the constraint, and λ is the Lagrange multiplier, it must hold that

$$\nabla f(\vec{x}) + \lambda \nabla g(\vec{x}) = \vec{0}$$

as well as $g(\vec{x}) = 0$.[13]

It is best to demonstrate this theorem with a simple calculus example, before we apply it to our mechanical problem.

Example 2.4.1. Assume $f(x, y) = xy$ and $g(x, y) = x^2 + y^2 - 1 = 0$. First let's calculate the necessary gradient vectors,

$$\nabla f(x, y) = \begin{pmatrix} y \\ x \end{pmatrix}$$

$$\nabla g(x, y) = \begin{pmatrix} 2x \\ 2y \end{pmatrix}.$$

Following the theorem, we then introduce this yet unknown Lagrange multiplier λ and say

$$\begin{pmatrix} y \\ x \end{pmatrix} + \lambda \begin{pmatrix} 2x \\ 2y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

The trick now is to get a system of equations out of this, which we can use to eliminate λ and be left with a relation between x and y. Here we get

$$\begin{cases} y + 2\lambda x = 0 \\ x + 2\lambda y = 0 \end{cases}$$

from which it can be derived that either $x = y$ or $x = -y$. The final step is to enforce the constraint, so $g(x, y) = 0$ is the last thing left to be fulfilled. Performing these final steps

results in four possible optimal points,

$$\left(\frac{1}{\sqrt{2}},\frac{1}{\sqrt{2}}\right),\left(-\frac{1}{\sqrt{2}},\frac{1}{\sqrt{2}}\right),\left(\frac{1}{\sqrt{2}},-\frac{1}{\sqrt{2}}\right),\left(-\frac{1}{\sqrt{2}},-\frac{1}{\sqrt{2}}\right).$$

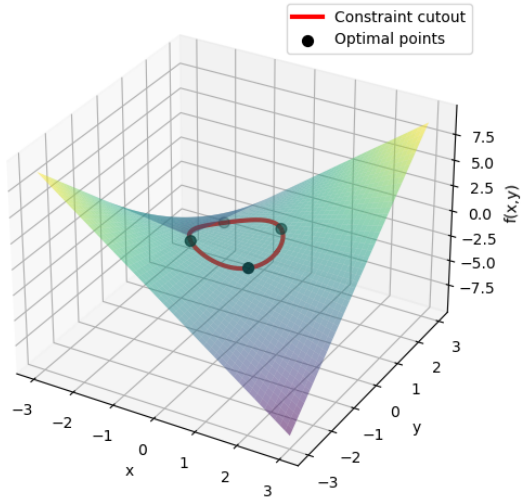


Figure 2.1: Visualization of example 2.4.1

Chapter 3

Constraints as linear algebra

3.1 Gauss's principle in matrix form

It is time to convert Gauss's principle and the aforementioned constraints into their linear algebra form. Since this is an important step, we'll take our time with it. First, let's recall the quantity Gauss defined to be minimized,

$$Z = \frac{1}{2}\sum_{i=1}^{n} m_i \left| \ddot{\vec{r}}_i - \frac{\vec{K}_i}{m_i} \right|^2 .^1$$

To express the masses in our system, we will require a block-diagonal mass matrix M. In two dimensions it can be defined as

$$M = \mathrm{diag}(m_1 I_2, m_2 I_2, \ldots) = \begin{pmatrix} m_1 & 0 & 0 & 0 & \cdots \\ 0 & m_1 & 0 & 0 & \cdots \\ 0 & 0 & m_2 & 0 & \cdots \\ 0 & 0 & 0 & m_2 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

which is a matrix with dimensions $2N \times 2N$.

Next is the actual acceleration. Both the final acceleration - the thing we're looking for - and the initial acceleration due to known forces - equivalent to $\frac{\vec{K}}{m}$ - will be written as column vectors. In two dimensions this becomes

$$\vec{a} = \begin{pmatrix} \vec{a}_1 \\ \vec{a}_2 \\ \vdots \end{pmatrix} = \begin{pmatrix} a_{1x} \\ a_{1y} \\ a_{2x} \\ a_{2y} \\ \vdots \end{pmatrix}$$

which is a vector of length 2N. Also note that for convenience purposes we will stop drawing the vector arrow in notation from now on. We can then redefine Gauss' quantity as

[1] The 1/2 factor was not present before. It is a standard in this formulation, so we will use it. Note however that the minimalization is not impacted by a constant factor.

$$Z = \frac{1}{2}(a - a_0)^T M(a - a_0).$$

The two terms on the right form another column vector of length 2N, that being $a - a_0$ but each term correctly multiplied by the according mass. Adding on the leftmost vector multiplication, it's the transposition that makes this turn into a scalar at the end. That scalar being the exact same thing we defined in Gauss's principle earlier.

3.2 Constraints in matrix form

Next we need to convert our constraints to a matrix form. Suppose we have k unique constraints, given by

$$\begin{cases} \phi_1(\vec{r}_1, \vec{r}_2, \ldots) = 0 \\ \phi_2(\vec{r}_1, \vec{r}_2, \ldots) = 0 \\ \qquad\vdots \\ \phi_k(\vec{r}_1, \vec{r}_2, \ldots) = 0 \end{cases}.$$

We already know that differentiating these twice results in the proper constraint equations involving acceleration terms - or if we're performing numerical simulations we also do Baumgarte stabilization. In the end this will result in k equations with a total of 2N variables - acceleration in both directions per particle. The entirety of the constraint problem will be summarizable via

$$Aa = b$$

where A is a matrix of dimensions $k \times 2N$ and b is a column vector of length k.

Example 3.2.1. To demonstrate, let's assume two particles are connected by a massless rod of some fixed length L. This will be covered as one of the examples in chapter 5. The eventual expression we get for the constraint is

$$C(\vec{a}_1, \vec{a}_2) = (\vec{r}_2 - \vec{r}_1) \cdot (\vec{a}_2 - \vec{a}_1) + (\vec{v}_2 - \vec{v}_1)^2 = 0.$$

This is a single constraint, so $k = 1$, but with two particles our position, velocity, and acceleration vectors each have a length of 4. It might not be fully obvious what the correct matrix A is, since vectors are involved, so let's write this one out:

$$(x_2 - x_1)(a_{2x} - a_{1x}) + (y_2 - y_1)(a_{y2} - a_{y1}) = -(\vec{v}_2 - \vec{v}_1)^2$$

Now it should be more obvious, our matrix is equal to

$$A = \begin{bmatrix} -(x_2 - x_1) & -(y_2 - y_1) & (x_2 - x_1) & (y_2 - y_1) \end{bmatrix} = \begin{bmatrix} -(r_2 - r_1)^T & (r_2 - r_1)^T \end{bmatrix}$$

and for the vector - which in this case is just a scalar - we find

$$b = -(\vec{v}_2 - \vec{v}_1)^2.$$

3.3 Lagrange multipliers for multiple constraints

As before, the next step is to solve a constrained optimization problem. The coordinates of the problem are all those final accelerations, or in linear algebra terms simply the vector a. We covered the basic Lagrange multiplier theorem for one constraint, but it can be easily expanded to multiple constraints. We simply introduce an equal amount of Lagrange multipliers, one per each of the k constraints.

Previously the general problem with one constraint and n coordinates - n is the length of $\vec{x}$ - could be solved by working out the system

$$\begin{cases} \nabla f(\vec{x}) + \lambda \nabla g(\vec{x}) = \vec{0} \\ g(\vec{x}) = 0 \end{cases}$$

which involves $n+1$ coordinates - the entirety of $\vec{x}$ plus λ - and a total of $n+1$ equations - the gradients create n and then one for the constraint.

Theorem 3.3.1 (General Lagrange Multiplier Theorem). To solve a constrained optimization problem, where $f(\vec{x})$ is the function to minimize/maximize, $\{g_1(\vec{x}), \ldots, g_k(\vec{x})\}$ is the set of constraints to satisfy, and $\vec{\lambda}$ is the vector of length k containing the Lagrange multipliers, it must hold that

$$\begin{cases} \nabla f(\vec{x}) - \lambda_1 \nabla g_1(\vec{x}) - \ldots - \lambda_k \nabla g_k(\vec{x}) = \vec{0} \\ g_1(\vec{x}) = 0 \\ \vdots \\ g_k(\vec{x}) = 0 \end{cases}$$

[13]

Once again, this is a system with $n+k$ unknowns and $n+k$ equations.

3.4 KKT matrix

In our specific case, the coordinates are all the elements of the vector a, and the constraint is given by $Aa - b = 0$, so one equation for each of the k rows. Due to the shape of these constraints, it shouldn't be a surprise that the gradients we're looking for are just the rows of A. Take for example the first constraint, which corresponds to the first row of A and first element of b. We can write it plainly as $A_{1,1}a_{1x} + A_{1,2}a_{1y} + \ldots + A_{1,2N}a_{ny} - b_1 = 0$, now taking the gradient of this simply gives us $[A_{1,1}\ A_{1,2}\ \ldots\ A_{1,2N}]$, which is indeed the first row of the matrix A. The gradient of the function to minimize, which is $Z = \frac{1}{2}(a - a_0)^T M(a - a_0)$, is given by $M(a - a_0)$. Therefore we must solve the system

$$\begin{cases} M(a - a_0) - A^T\lambda = 0 \\ Aa = b \end{cases}$$

by first finding that

$$a = a_0 + M^{-1}A^T\lambda$$

and plugging into the second equation to find

$$A(a_0 + M^{-1}A^T\lambda) = b$$

$$(AM^{-1}A^T)\lambda = b - Aa_0.$$

For our case, the system can also be written as

$$\begin{bmatrix} M & -A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} a \\ \lambda \end{bmatrix} = \begin{bmatrix} Ma_0 \\ b \end{bmatrix}$$

in which the matrix is known as the Karush-Kuhn-Tucker matrix. Matrices like these - symmetrical, positive and often diagonally dominant - always show up when solving constrained optimization problems in multiple coordinates.

Chapter 4

Numerical constraint solvers

If we continue directly from the previous chapter, we now need to solve the linear system given by

$$(AM^{-1}A^T)\lambda = b - Aa_0$$

which results in a solution for λ that can then lead to the eventual final accelerations via

$$a = a_0 + M^{-1}A^T\lambda.$$

4.1 Direct VS iterative solving

The weigh-off needs to be made between direct solving methods and iterative solving methods.

Direct solving methods, like Gauss elimination combined with backward substitution, will give an exact solution up to errors caused by numerical stability and rounding.

Iterative solving methods start with a guess for the solution and use a fixed iteration technique to inch closer and closer to the exact solution, only stopping once they're close enough. Though it sounds worse than just solving directly, these methods are often much faster in their computational complexity and can be tweaked to find the right balance between computing time and accuracy. Furthermore, methods like impulse-based solving incorporate the time-stepping part of the simulation directly into the constraint solving. This highlights why iteration isn't so bad after all: isn't taking steps in time exactly that?

4.2 Gauss-Seidel method

The Gauss-Seidel method is an iterative method that primarily focuses on solving systems of non-linear equations. In other words, it strives to find the vector $\vec{x}$ of length n for which all functions $f_1(\vec{x}), \ldots, f_n(\vec{x})$ are equal to zero. The way it works is simple: after having chosen a starting point for the iteration, we first approximate the vector function using the Jacobian - a

sort of higher dimension Taylor series around the current iteration point $\vec{x}^{(k)}$.

$$\vec{y}(\vec{x}) = \vec{f}(\vec{x}^{(k)}) + \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{pmatrix} (\vec{x} - \vec{x}^{(k)})$$

This large matrix of partial derivatives, all of which should be evaluated in $\vec{x}^{(k)}$, is called the Jacobian and will be referred to as J or $J^{(k)}$ from now on. The actual iteration step says that our next point should be the one where $\vec{y}(\vec{x}) = 0$. In the one-dimensional case, you can imagine this function to be the tangential line to f at $x^{(k)}$, and the intersection between this line and the x axis is your new point. Thus we find the expression

$$J(\vec{x}^{(k+1)} - \vec{x}^{(k)}) = -\vec{f}(\vec{x}^{(k)}).$$

What we've done so far is called the Newton-Raphson method. If you're performing that algorithm, you'd stop here and find the next point $\vec{x}^{(k+1)}$ by solving this linear system. But this doesn't help us! We're trying to use this technique to solve an actual linear system iteratively, and if we were to plug it into this, it would just force us to use (slow) Gauss elimination - a direct method! So we need to use an approximated version of this technique. It involves using only the diagonal of your Jacobian and setting all the other elements to zero.

In this specific case, you can rewrite the step as

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} - J^{-1}\vec{f}(\vec{x}^{(k)})$$

and since J is now diagonal - the inverse literally being the same with each element inverted - you can perform the step per coordinate as

$$x_i^{(k+1)} = x_i^{(k)} - \frac{f_i(\vec{x}^{(k)})}{\frac{\partial f_i}{\partial x_i}(\vec{x}^{(k)})}.$$

An important thing to note is that the order in which you number the different functions f_i determines which partial derivatives will land on the diagonal. While it may seem unimportant, a different set of partial derivatives may lead to convergence becoming divergence or vice versa. But the real Gauss-Seidel method goes even further - this is just the Jacobi method. The above equation is performing the step for the i-th coordinate of $\vec{x}$, so we have already stepped coordinates $1, \ldots, i-1$. Why not already use those new coordinates in the function and derivative calls, like below?

$$x_i^{(k+1)} = x_i^{(k)} - \frac{f_i(x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}, x_i^{(k)}, \ldots, x_n^{(k)})}{\frac{\partial f_i}{\partial x_i}(x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}, x_i^{(k)}, \ldots, x_n^{(k)})}$$

Doing this will greatly increase the speed of convergence, since new information is being used more rapidly. On the other hand, we've introduced yet another annoying degree of freedom: the order in which to step the coordinates. If you were just doing Jacobi iteration, this wouldn't matter, since you keep using the old coordinates to evaluate. But since you're using the coordinates you've already stepped to calculate the next one, the order now matters! A different order can imply a whole different level of accuracy or straight-up divergence. This however is not as

fragile as the function ordering from before.

Now that the method has been covered, let's apply it to our linear system.

$$(AM^{-1}A^T)\lambda = Aa_0 - b$$

For the sake of readability, let's define $S = AM^{-1}A^T$ and $r = Aa_0 - b$. This reduces the problem to solving $S\lambda = r$, where S is a matrix of dimensions $k \times k$ and r is a vector of size k. The reason why we can even use Gauss-Seidel on linear systems is because $S\lambda = r$ is the same as $S\lambda - r = 0$, which defines a linear systems, or k functions that should be zero at our λ. The Jacobian here is literally just S itself, which makes things a lot simpler. We get

$$\lambda_i^{(k+1)} = \lambda_i^{(k)} - \frac{\sum_{j=1}^{i-1} S_{ij}\lambda_j^{(k+1)} + \sum_{j=i}^{k} S_{ij}\lambda_j^{(k)} - r_i}{S_{ii}}$$

4.3 Impulse-based simulations

The next step is to completely get rid of accelerations and perform our system solving on a velocity level, such that we can incorporate it directly into the time steps.

Like we did Verlet-style, we update velocities via

$$v^{n+1} = v^n + \Delta t a.$$

We know what a is from solving multi-dimensional Lagrange multipliers, so we get

$$v^{n+1} = v^n + \Delta t a_0 + \Delta t M^{-1}A^T\lambda.$$

Now we'll define some terminology to shorten things, like

$$v_0^{n+1} = v^n + \Delta t a_0$$

which is called the unconstrained velocity, just like a_0 was the unconstrained acceleration - only including those known or external forces. We also define the core of this section, the constraint impulse J - not the Jacobian! - as $J = \Delta t\lambda$. It's quite literally a jump from unknowns in the acceleration domain to unknowns in the velocity domain. We end up with

$$v^{n+1} = v_0^{n+1} + M^{-1}A^T J.$$

Using capital J as a symbol might make the constraint impulse seem like a matrix, but it is of course still a vector of length k.

What's most important to see now is that if we have our simple ideal, holonomic constraints, they will be expressed - or expressable - at a velocity level via

$$Av = 0$$

where v is the long vector of length 2N containing the velocities $v_{1x}, v_{1y}, v_{2x}, \ldots$ and A is actually the same matrix as we had for acceleration-based constraints. This makes sense, as deriving $Av = 0$ once more to get to the acceleration level would yield two terms due to the product formula: Aa and b, where b is created by the derivatives of the coefficients that were next to

the elements in v. Take for example the rigid rod example from before. At a velocity level it is expressed as

$$2(\vec{r}_2 - \vec{r}_1) \cdot (\vec{v}_2 - \vec{v}_1) = 0$$

such that the matrix A (1×4) would once again become

$$A = \begin{bmatrix} -(x_2 - x_1) & -(y_2 - y_1) & (x_2 - x_1) & (y_2 - y_1) \end{bmatrix} = \begin{bmatrix} -(r_2 - r_1)^T & (r_2 - r_1)^T \end{bmatrix}.$$

Taking the derivative again yields both Aa but also includes the derivative of $\vec{r}_2 - \vec{r}_1$, which forms the b vector we saw at the acceleration level.

So at each time step, the new velocity - which has been expressed according to Gauss's principle and a so far unknown J - must respect these velocity-level constraints, meaning

$$A\left(v_0^{n+1} + M^{-1}A^T J\right) = 0$$

$$(AM^{-1}A^T)J = -Av_0^{n+1}$$

which is the equation to solve for J, preferably using Gauss-Seidel. So to recap, here's how we got here:

- Gauss's principle tells us how constraints should minimally affect the unconstrained acceleration.
- Using multi-dimensional Lagrange multipliers, we were able to find the expression for the constrained acceleration, which contains an unknown λ found by respecting the constraint $Aa = b$. This equation follows directly from the fact we're minimizing Gauss's quantity Z, so using it anywhere satisfies the principle.
- If we were to stop here, we would use Gauss-Seidel to solve for λ, calculate the constrained acceleration, and apply it for a velocity time step.
- But we can make this faster by skipping the acceleration calculation. Instead, we use our previous expression for the constrained acceleration to directly define what v^{n+1} would be, and use velocity-expressed constraints to solve for J - which is just λ in a different suit.
- This not only skips some steps, but also slightly increases the efficiency of the Gauss-Seidel solve, since there is no more b in our constraint expression.

4.4 Projected Gauss-Seidel (PGS)

The final step in working out the core of mechanical simulations is to mix Gauss-Seidel directly with incremental fixes in the velocity. The grand scheme is something like this, performed each time-step:

- Calculate the unconstrained velocities, by using the previous velocity and the unconstrained acceleration from external forces like gravity, springs, ...
- Initiate the constraint impulse - our unknown - as $J = 0$.
- Perform Gauss-Seidel, meaning we do multiple full updates of J.
- Within each GS iteration, we're updating every J_i one by one - and using the new items ($j < i$) instantly.

- We can rewrite GS to tell us what ΔJ_i will be, so that we can update J but also instantly update the velocity v with only the impact from J_i changing. We do this because our rewritten version of GS will refer to the (constrained) velocity itself in calculation - so the next J_{i+1} needs the new, more correct version.
- When we have performed enough full GS iterations, each one itself consisting of an update to each J_i along with an update to v, we should have a proper numerical estimation of the constrained velocity.
- Finally just use $x^{(i+1)} = x^{(i)} + v\Delta t$ to take one step in time for positions.
- Continue to the next time-step.

Before we get into the mathematical details, it's important to note how doing it this way makes much more sense when viewed from the constraint perspective. As you'll see, the calculation for ΔJ_i and equivalently the immediate update for v, will only involve row i of our matrix A (A_i), which corresponds exactly to one constraint in our system: the one that J_i is linked to. So we won't construct the whole matrix anymore, it makes more sense now to reason from constraint to constraint - row to row, J_i to J_{i+1} - and is also much more efficient.

Let's derive the actual proper calculations. Begin by remembering what we're working with, which is

$$v^{n+1} = v_0^{n+1} + M^{-1}A^T J$$
$$(AM^{-1}A^T)J = -Av_0^{n+1}.$$

If we perform Gauss-Seidel for this latter system, we are saying

$$J_i^{(new)} = J_i^{(old)} - \frac{1}{S_{ii}}\left(\sum_{j=1}^{k} S_{ij}J_j - r_i\right).$$

Here it's important to note that the sum is no longer split up into the $j < i$ and $j \geqslant i$ parts. This is because we're instantly updating the J vector, so simply referencing J_{i-1} already gives the updated value - from the previous step - and further elements are the old ones, yet to be overwritten. Now let's take a closer look at S and r in this case. We can see that

$$S_{ij} = A_i M^{-1} A_j^T$$

where A_i is row i of the matrix A. The rightmost term is equivalent to column j of A^T, which is indeed just the transpose of row j of the matrix A. It is trivial to check that given this form, S_{ii} in the denominator will always be non-zero. We also know

$$r_i = -A_i v_0^{n+1}$$

such that the update becomes

$$J_i^{(new)} = J_i^{(old)} - \frac{A_i v_0^{n+1} + A_i M^{-1} \sum_{j=1}^{k} A_j^T J_j}{A_i M^{-1} A_i^T}. \tag{4.1}$$

Now let's take a closer look at the constrained velocity expression and see how we can introduce this term into our constraint impulse update, such that the velocity can be iterated too. Note that the equivalence

$$A^T J = \sum_{j=1}^{k} A_j^T J_j$$

holds and thus we can say

$$M^{-1}\sum_{j=1}^{k} A_j^T J_j = v^{n+1} - v_0^{n+1}.$$

Plugging into eq. (4.1) then gives

$$J_i^{(new)} = J_i^{(old)} - \frac{A_i v^{n+1}}{A_i M^{-1} A_i^T}$$

or equivalently - and more usefully -

$$\Delta J_i = -\frac{A_i v^{n+1}}{A_i M^{-1} A_i^T}. \tag{4.2}$$

This seemingly simple fraction gives us the change in J_i for this step of this iteration of Gauss-Seidel. We then use it to update the J vector and also to update our constrained velocity. The latter can be done via

$$v^{n+1} = v_0^{n+1} + M^{-1}\sum_{j=1}^{k} A_j^T J_j$$

where if only one coordinate J_i changes, the influence on v^{n+1} is

$$\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i.$$

If we wish to further increase the efficiency of the technique, we could try to warm-start our J vector. Right now we reset $J = 0$ at each time-step, before starting the GS iterations. But if the time-steps are small enough, we can expect minimal changes to J each time, so it'd be useful to warm-start $J_0^{(k+1)} = J^{(k)}$ and let it do the few remaining iterative changes from there. This causes faster convergence and saves on computation. We do have to be wary about the velocity here. If you say $J_0 = 0$, then initiating $v^{n+1} = v_0^{n+1}$ - the unconstrained velocity - and applying $\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i$ per update works perfectly. But if you start with a non-zero J, you also can't start with the velocity being just the unconstrained velocity, but rather $v^{n+1} = v_0^{n+1} + M^{-1} A^T J$ and only then can you start applying the correct tiny updates!

Warm-starting comes with its downsides though. It's a great method to increase efficiency by remembering the past, but what happens if sudden changes in the system occur? Say you have two balls stacked on top of each other and you drop a huge third ball on top of them. The system will stabilize and converge thanks to warm-starting - even if your amount of iterations is small - but if you now remove the third huge ball, you'll notice the top ball bouncing up. This is because in the first frame where the huge ball is gone, we warm-start J using the one from the previous frame, when the ball was still present. And if your amount of iterations is small, there won't be enough opportunity for J to adapt to the new situation, so inconsistencies like the bouncing occur.[3]

4.5 Incorporating contacts

All the constraints we've covered so far are holonomic, because they can be expressed as an analytical relation, like $Av = 0$. This however will not always be the case, especially when

dealing with contacts.

Contact is a posh name for what will basically be collisions between objects in the simulation. While this might only make you imagine two balls smashing into each other, technically an object sliding on a floor or bouncing on a stationary wall is also considered a contact. Contacts are modeled via inequalities, such as $Av > 0$. But unlike permanent connections or holonomic constraints between particles, these inequality constraints are introduced in a different way. The contact constraint between two objects should only be added to the list of constraints - equivalently the A matrix - if those objects are actually colliding. The system that detects these overlaps is completely seperate from the dynamical solver, and we will cover it later. For now, just imagine that at the start of each time-step, some magical function is able to tell us which particles are colliding.

For each pair of colliding particles - or objects - one contact constraint will be added, or one row in A. If we imagine this being a collision between object i and object j, whose velocities appear in v at index $2i$ respectively $2j$, we would add this row:

$$\begin{bmatrix} 0 & \dots & n_x & n_y & \dots & -n_x & -n_y & \dots & 0 \end{bmatrix}, \quad n_x \text{ at position } 2i,\ -n_x \text{ at } 2j$$

Where $\vec{n} = (n_x, n_y)$ is the normal vector pointing from particle j to particle i. Now let's see what calculating $A_k v$ - k representing this row - would equate to.

$$A_k v = \vec{n} \cdot \vec{v}_i - \vec{n} \cdot \vec{v}_j = \vec{n} \cdot (\vec{v}_i - \vec{v}_j)$$

This is a relative velocity projected in the normal direction. More specifically, the right-hand term represents the velocity of i from the perspective of j. So if this expression is positive, then from object j's perspective, object i is moving in the direction of $\vec{n}$, so away from j. For objects that are colliding, we want to constrain their velocities such that they move apart relatively, so clearly what we need to enforce with this constraint is

$$A_k v > 0.$$

Mathematically, both the equality and inequality constraints we've covered can generally be called KKT conditions[14]. For equality constraints the solving method is equivalent to what we covered with Lagrange multipliers. For inequality constraints, we also introduce Lagrange multipliers, but the conditions they must satisfy are slightly different. The same update from eq. (4.2) applies, and the velocity still changes depending on how J_k changes, but we also enforce

$$\begin{cases} J_k > 0 \\ A_k v \cdot J_k = 0 \end{cases}.$$

The first statement is an obvious one. If you look at the velocity update equation

$$v^{n+1} = v_0^{n+1} + M^{-1} \sum_{j=1}^{k} A_j^T J_j$$

you'll see that if J_k is positive, the velocity increase - basically a force - on object i is in the direction of $\vec{n}$, so away from object j. In other words, enforcing $J_k > 0$ assures the created constraint force - which could be like a normal force - is always pushing objects apart.
As for the second statement, let's look at two main situations to observe why this is true.

- Option 1: $A_k v_0 > 0$

  This means that at the start of our iterations, even though the objects are colliding, they are already moving apart and thus satisfying the constraint. According to eq. (4.2) this will cause J_k to decrease and decrease, until it reaches zero where it is clamped because of the $J_k > 0$ requirement. In the end, this means the constraint has no effect, does not attribute to a velocity change and thus no forces - we call the constraint inactive. Since this situation has pushed to $J_k = 0$, the second statement does indeed hold.

- Option 2: $A_k v_0 < 0$

  This means that at the start of our iterations, the objects are trying to penetrate each other even more, since their relative velocities do not satisfy the "moving apart" constraint. According to eq. (4.2) this will cause J_k to increase and increase, applying a pushing force to the particles, until the constraint is minimally satisfied, or $A_k v = 0$. At that point, ΔJ_k will also be 0 and since there will be no more changes, the second statement does indeed hold.

So in practice we can implement contacts as follows:

- Use an external method at the start of a time-step to list which pairs of objects are colliding.
- For each of these pairs, an inequality constraint is added to the system that time-step.
- When encountering these constraints in the iteration, calculate and apply ΔJ_k as always. However do not apply it to the velocity just yet.
- First clamp J_k to be positive, often by saying $J_k = \max(0, J_k + \Delta J_k)$.
- Calculate the actual change $\Delta J_{k,real} = J_{k,old} - J_{k,new}$ which can differ from ΔJ_k if clamping was performed. For example, if J_k = 3 and $\Delta J_k = -5$, then the resulting $J_k = 0$ and thus $\Delta J_{k,real} = -3$.
- Apply the velocity update using this actual change, so $\Delta v^{n+1} = M^{-1} A_k^T \Delta J_{k,real}$.

Some might still be wary of this implementation, since the way we actually implemented the new constraint, with the J_k updates, might make it look like we're still enforcing $Av = 0$ instead of $Av > 0$. You're right, theoretically we are enforcing the equality. But we also clamp $J_k > 0$, which limits which forces can be used to make the constraint respected. In this case it only allows pushing forces, so the case where $Av < 0$ will be repaired until $Av = 0$, but if you already have $Av > 0$ then technically speaking it would do ΔJ_k until eventually $J_k < 0$ such that it could start working towards $Av = 0$. However we clamp J_k to be positive, so in this case it actually just stays at 0 and leaves the already satisfied constraint alone. It's this equality constraint combined with a clamp on the constraint impulse that lets us control the implied forces, which will become more evident with friction.

4.6 Incorporating friction

Friction between two objects happens if they have tangential relative velocity. It acts up to a certain maximum to reduce this velocity - this determines the direction. Friction happens between any two objects in contact, so it can be added alongside the actual contact constraint. This time, we will try to enforce the constraint saying 'no relative tangential velocity' using a new row in A. However, we will clamp J_k to some interval, such that the force trying to reduce

the tangential relative motion has a maximum, just like real friction. This interval's size should also depend on the normal force, caused by the constraint we covered in the previous chapter, so it is controlled by the eventual - or iterated - J_k for that constraint.

First let's take a look at the new row we need to introduce into A. As we just covered it should represent the relative tangential velocity as Av. So the row is

$$\begin{bmatrix} 0 & \dots & t_x & t_y & \dots & -t_x & -t_y & \dots & 0 \end{bmatrix}, \quad t_x \text{ at position } 2i,\ -t_x \text{ at } 2j$$

where $\vec{t} = (t_x, t_y)$ is the tangent vector, so the vector perpendicular to $\vec{n}$ from before. Technically speaking, on a 2D grid, there are two possibilities for such a vector. But it does not matter which one you use, the sign of Av and J_k will flip accordingly, always causing an opposing force. So now $A_k v = \vec{t} \cdot (\vec{v}_i - \vec{v}_j)$, the velocity of i from j's perspective, projected on the tangent vector.

Enforcing $A_k v = 0$ means the system we already have with J_k updating and gradually creating a force that changes v, will try to get this relative velocity down to zero. But friction has a maximum. J_k can be either positive or negative, whichever is needed to reduce the relative motion, but its magnitude should be fixed. In other words, we have to clamp

$$J_k \in [-\mu |J_{normal}|, \mu |J_{normal}|]$$

where J_{normal} is the constraint impulse that corresponds to the contact constraint for these same two colliding objects. This will guarantee that the highest possible absolute effect on velocity is $\Delta\vec{v}_i = \frac{\vec{t}\mu|J_{normal}|}{m_i}$, which is indeed equivalent to the way friction is usually calculated: some multiple of the normal force (in magnitude).

So once again this is introduced into the loop by calculating ΔJ_k as normal, updating J_k with it, clamping it to this interval, then calculating the real change and applying that to the velocity update. Since the iterations of J_k and J_{normal} are happening side-by-side, this gives rise to more complex interactions between constraints.

The keen-eyed among you might be terribly upset right now, because this system clearly doesn't distinguish between static and dynamic friction. You are right. Incorporating this differentiation would require you to take a look at the beginning $A_k v$, and testing if the maximum static friction - something like $J_k = \pm\mu_{static} |J_{normal}|$ - is sufficient to completely bring $A_k v$ down to zero. If it is not, then you know relative motion will still occur and thus you should switch to dynamic friction instead, where the dynamic μ is often smaller than the static one. However this requires you to already know the final iterated J_{normal}, and the proper transitioning from static to dynamic calculations - without stuff blowing out of proportions - is extremely hard to control. This is why real simulations do not make a distinction between the two types and just take a μ that averages the two. And to be honest, barely anybody would be able to tell the difference between the implementations.

4.7 Baumgarte stabilization

Let me introduce you to a shocking truth: as beautiful as these numerical iteration methods are, they remain numerical and thus fundamentally introduce errors into the system. Another truth is that our current iterations focus on velocity-based constraints. Take for example the fixed distance constraint, where we shove a rod of some constant length L between two particles or

objects. To implement this, we'd use the velocity-based constraint, $2(\vec{r}_2 - \vec{r}_1) \cdot (\vec{v}_2 - \vec{v}_1) = 0$. This way, the system will try to orient velocities using constraint forces such that the connected objects don't lengthen or shorten the distance between each other.

But this won't go without some numerical errors, either due to rounding or due to a limited amount of Gauss-Seidel iterations. These errors accumulate throughout the simulation and can cause the length of our rod to lose track of its original L. And since this velocity-based constraint carries no information about that original length, the system has no idea something is going wrong. The rod acts sort of like a grandma with dementia: she does her best to keep all the money in her purse, but she can never remember what the exact amount she should have is, and she fails to notice thieves stealing nickles and dimes from her every Δt.

The solution is to keep everything as it is now, apart from the solving equation for J. This equation arose from combining the expression for v with $Av = 0$. So now we'll instead define

$$Av = -\frac{\beta}{\Delta t} C(x).$$

[1] This makes the constraint act as more of a spring-dampened system, which now includes information about the positions x and some constraint C on them, telling the system to recoil back. We call $\frac{\beta}{\Delta t} C(x)$ the bias. The constraint we're trying to enforce on the positions here is $C(x) = 0$. So for the rod example the corresponding $C_k(x)$ would be $(\vec{r}_i - \vec{r}_j)^2 - L^2$, zero if the distance is properly conserved.

For the contact constraint, you might enforce there to be no penetration, so $C_k(x)$ represents the gap between the objects along the $\vec{n}$ direction. If it is negative, they are overlapping, if it is positive, they are already apart. But here you must once again make a distinction! We're going to try and enforce $C_k(x) = 0$, but if it is already positive then they're already apart and we need not enforce anything. So in reality you would say the bias is equal to $\frac{\beta}{\Delta t} \min(0, C(x))$.

For the friction constraint, you don't need to enforce anything for positions. All this constraint does is implement friction by reducing relative tangential velocity up to some maximum. No positional information is involved, so here $C_k(x) = 0$ at all times.

With this new expression we need to re-derive the solving system for J, as well as the Gauss-Seidel update equation. Luckily barely anything actually changes. We still arrive at

$$v^{n+1} = v_0^{n+1} + M^{-1} A^T J$$

and now use

$$Av = -\frac{\beta}{\Delta t} C(x)$$

to find the equation for J as

$$\left(AM^{-1}A^T\right) J = -Av_0^{n+1} + D$$

where we've defined

$$D = -\frac{\beta}{\Delta t} C(x).$$

The same steps as before also apply to implementing a Gauss-Seidel update. We once again fill in the values and through some substitutions find the update expression that involves the

current iteration of the constrained velocity. Only now we also have D in there, as

$$\Delta J_i = -\frac{A_i v^{n+1} - D_i}{A_i M^{-1} A_i^T}.$$

We can rewrite this to its final form

$$\Delta J_i = -\frac{A_i v^{n+1} + \frac{\beta}{\Delta t} C_i(x)}{A_i M^{-1} A_i^T}$$

and remember the same velocity update still holds,

$$\Delta v^{n+1} = M^{-1} A_i^T \Delta J_i.$$

4.8 Nonlinear Gauss-Seidel (NGS)

The solver as described up until this point would be perfectly capable of yielding good-looking, stable simulations. Its accuracy towards constraints however, is less desirable. If you run a distance constraint test simulation with Baumgarte stabilization, you'll notice the positional constraint it is supposed to enforce, $d - L = 0$, isn't zero but a small decimal value depending on your settings. The problem here is that the enforcing of Baumgarte stabilization, which right now is the only thing actually reminding the system of its positional accuracy, is mixed with updating an already informed velocity. We start with a velocity that is carried over from the last frame and already has external forces applied to it. Then the velocity-based constraint has to fight with the Baumgarte term in the numerator over ΔJ. Eventually it settles and the constraint is satisfied, but we pollute the next frame with positional corrections. A better approach would be to separate the positional constraint from the velocity one.

So we'll remove the Baumgarte term in the numerator of our velocity iteration, and run that like before. This yields an updated (final) v^{n+1}. Before we apply that velocity for a positional step in time, we let a separate, but similar positional solver do its job to correct positional violations of the constraint. It uses a pseudo-velocity v^* - stored per body just like normal velocities and also including rotational velocity, initiated at zero such that it carries no conflicting information. Using the same Gauss-Seidel updates, but including the Baumgarte term in the numerator, we can calculate ΔJ and find out what tiny velocity might be able to resolve the violation. We still perform this multiple times, say N, for each constraint, but after each loop we already apply the pseudo-velocity to positions (over $\frac{\Delta t}{N}$) and reset it back to zero. This allows the system to converge using multiple small adjustments. While the resulting remaining positional error will never be zero - simulations always remain numerical - they will be orders of magnitude smaller and the simulation will be more stable. Tinkering with the amount of iterations (N), the positional constraint stiffness (β) and the time-step detail (Δt) allows us to minimize the error even more, at the cost of computation and - for extreme values - stability.

4.9 Further optimizations

In practice, more advanced techniques or completely different approaches can be used for specific applications to yield results with less numerical error and more stability. We will call it quits here, but the interested reader can always find more information via the documentation of popular physics engines such as Box2D and Bullet2D.

Chapter 5

Implementing rotation

So far we have completely neglected the possibility of rotation in our simulation. The objects we currently have are just controlled by translational movements, seen from the mass center - it's easy to mathematically prove that forces, speeds, and momentum can all be regarded from the mass center. But rotations are necessary, so we'll incorporate them.[5]

5.1 New state vector

Since we're looking at 2D motion, rotation only adds one degree of freedom. If you imagine a circle on the screen right now, it's rotational axis would be going in or out of your screen, along some magical axis not on our 2D grid. We will update the state vector to include an angle of rotation θ. How this angle is defined does not matter, since any constant difference between two implementations has no impact on the speed, their derivatives.

$$\vec{x}(t) = \begin{pmatrix} x_1(t) \\ y_1(t) \\ \theta_1(t) \\ x_2(t) \\ y_2(t) \\ \theta_2(t) \\ \vdots \end{pmatrix}$$

5.2 Gauss's principle with rotation

Gauss's principle also needs to be updated to deal with rotation. Once again its premise will be that any acceleration changes caused by constraints should be minimally invasive. For translations, these accelerations (squared) were weighted by mass. Well as you might expect for rotations, they're weighted by moment of inertia, I. These will thus need to be determined for every single object. But for simple objects like circles and squares, this is no big deal. In general, this contribution is written as

$$Z = \frac{1}{2}\left((a - a_0)^T M (a - a_0) + \sum_{i=1}^{n} (\alpha_i - \alpha_{i0})^T I_i (\alpha_i - \alpha_{i0})\right) \tag{5.1}$$

where α is the rotational acceleration - the second derivative of θ - and I_i is the inertia tensor. The latter is generally a symmetrical 3×3 matrix containing

$$\begin{pmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{pmatrix}$$

with $I_{xx} = \int_V (y^2 + z^2)\, dm$ - infinitesimal sum of distances from the x-axis - and alike for others. But we don't need all that in 2D! Our α_i are not full vectors. If we define the x and y axes on our 2D grid, then the z axis is the one coming out of our screen. That's where the rotation lies, so we know $\alpha_i = (0 \quad 0 \quad \alpha_i)$ and thus the term inside the sum of eq. (5.1) becomes

$$(\alpha_i - \alpha_{i0})^2 I_{zz}.$$

In practice, when coding some object, we can calculate

$$I_{zz} = \iint_V (x^2 + y^2)\rho(x, y)\, dV$$

where the density $\rho(x, y)$ is a constant for homogeneous objects. It basically measures distance from the rotational axis weighted by mass.

To incorporate this, we only need to extend the mass matrix M to include I_{zz}, so

$$M = \begin{pmatrix} m_1 & 0 & 0 & 0 & \cdots \\ 0 & m_1 & 0 & 0 & \cdots \\ 0 & 0 & I_1 & 0 & \cdots \\ 0 & 0 & 0 & m_2 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}.$$

This combined with the state vector update means we don't even have to change anything about our methods. We write Z in the same way and thus everything else follows accordingly.

5.3 Points in bodies

When will these rotations actually come into play? Well we obviously need them for rendering the objects at their correct angles, but they also play a big role in our constraints. Those involve velocities, and as soon as we're not talking about the velocity of the mass center - where the rotational axis is - then the rotational velocity adds some component of speed, which might then cause a change in the angle.

More specifically, if we have some point i on an object and we wish to know its velocity, we need to sum the mass center velocity and the rotational contribution, via

$$\vec{v}_i = \vec{v}_{MC} + \vec{\omega} \times \vec{r}_i.$$

So here we need the cross product, between $\vec{r}_i$ - the vector from the origin to the point - and the angular velocity $\vec{\omega}$ - the derivative of θ stored in the velocity vector. This 'origin' should for any rotation always be somewhere along the rotational axis. So unsurprisingly in this sense the origin is just the object's mass center. And since $\vec{\omega} = (0 \quad 0 \quad \omega)$, we can simplify to

$$\vec{v}_i = \vec{v}_{MC} + (-y_i\omega \quad x_i\omega) \tag{5.2}$$

in our two dimensions, with x_i and y_i the components of $\vec{r}_i$, the vector pointing from the object's mass center to this point in its body.

5.4 Changes to holonomic constraints

With this in mind, our simple holonomic constraints can become a little more complicated. If we just take two circular objects and want the distances between their mass centers to be constant, then only translation is affected. However if you want to - more realistically - connect some points on the edges, you'll have to take rotation into account. Remember the positional constraint as

$$(\vec{r}_i - \vec{r}_j)^2 - L^2 = 0.$$

Deriving to find a velocity-based constraint requires the derivatives of both $\vec{r}_i$ and $\vec{r}_j$. In general, these are no longer just $\vec{v}_i$ and $\vec{v}_j$, but instead we'd get

$$(\vec{r}_i - \vec{r}_j) \cdot (\vec{v}_i + \vec{\omega}_i \times \vec{q}_i - \vec{v}_j - \vec{\omega}_j \times \vec{q}_j) = 0.$$

Do mind that I haven't expanded the cross products down to their simplest form like in eq. (5.2). We'll expand our equations when we actually need to implement them in chapter 8. What's important is that this expression, once you've extracted the terms for each factor $(v_{ix}, v_{iy}, v_{jx}, v_{jy}, \omega_i, \omega_j)$, will yield the constraint row in A, which is used in our iterative GS cycle to update v, which will now also tweak rotational velocity.

5.5 Changes to contact and friction constraints

Our contact and friction constraints also need to be updated. That's because so far I hadn't been very specific about which velocities to use there. Both constraints try to reduce down the relative velocity, projected perpendicularly or tangentially, with restrictions on J_k. But these velocities should always be for contact points!

While the dynamics solver (constraints, time updates, GS) is one component of our implementation, so is a completely seperate collision detection system. We'll cover this system in detail in the next chapter. What's important here is that this system will result in a big list of every colliding pair of objects, as well as the contact point(s) for these collisions. We then demand that for each contact point - which should be a point that exists on both objects - the relative velocity projection is zero, with the same restrictions. It involves calculating $\vec{q}$, the vector pointing from each object's mass center to the contact point, and then saying

$$(\vec{v}_i + \vec{\omega}_i \times \vec{q}_i - \vec{v}_j - \vec{\omega}_j \times \vec{q}_j) \cdot \vec{n} = 0$$

for the contact constraint. The friction case just involves a different projection. Everything else is the same. As shown in fig. 5.1, bigger contact surfaces require more contact points. This is to make sure the movement actually looks realistic. If we didn't add the points on the right and left, the upper box might clip into the lower box by rotating around the middle contact.

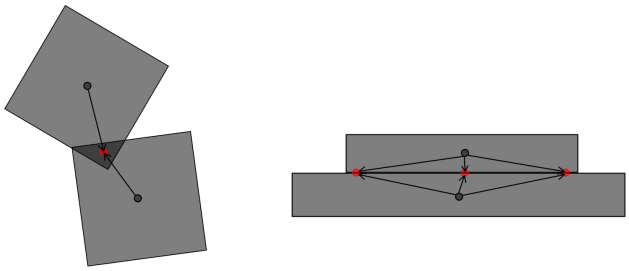


Figure 5.1: Two examples of possible collisions, the contact points (red), mass centers (gray) and relative position vectors

Chapter 6

Collision algorithms

Each time-step, before any dynamics solving is performed, we require a list of every active collision. In other words, a collision system that returns a list of body pairs where those pairs of bodies are overlapping. Per pair we also want the normal (and from that the tangential) vectors - for projection in constraints - as well as the penetration depth - for Baumgarte stabilization.

This system is comprised of two main parts:

- Broad-phase detection: performs a first, general inspection to rule out pairs of bodies that are obviously not colliding. Its eventual output will still contain false positives, but the goal is to bring down the number of pairs that need to be checked more thoroughly in the narrow phase.
- Narrow-phase detection: examines every potential colliding body pair - obtained from the broad phase - to determine with 100% accuracy whether a collision is taking place. This will therefore be a lot more computationally expensive, so the more the broad phase prunes, the better. A method used here might also allow us to efficiently obtain the normal/tangential vectors and the penetration depth. If not, this is easily calculated afterwards.

Efficient collision detection is an integral part to a fast physics engine, so it can be regarded as a research branch (in mathematics or topology) just by itself. We will try to do the mathematicians justice by looking into some of the most useful techniques.

6.1 Broad-phase algorithms

6.1.1 Sort, sweep, and prune

The sort, sweep, and prune algorithm uses a sorted list of the boundaries of bodies to detect possible collisions.

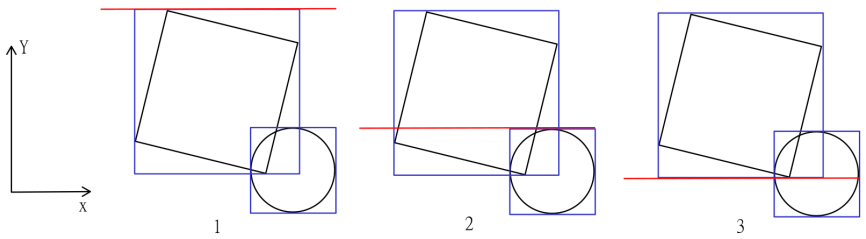


Figure 6.1: Three frames of the sweeping process between a square and a circle

The first step is to take all of your bodies and draw bounding boxes around them. These are rectangles, aligned with the given x- and y-axes, defined to be the smallest possible rectangles that completely encompass the specified body. These are shown in blue in fig. 6.1. The second step is to let a scanning line (here: red) run across the entire space for both directions. Whenever this line encounters the starting edge of a bounding box, the corresponding body is added to a tracking list. If the line encounters the ending edge, the body is removed. If at any point there are two or more bodies in this tracking list, they can be classified as potential colliding pairs. At **point 1** on the figure above, the square is added to the tracking list. At **point 2** the circle is added, while the square is still in there, which causes a collision detection. At **point 3** the square is removed from the tracking list.

Bounding boxes

To use the sweeping method, we need every object in our scene to be simplified to a simple rectangular shape. These are called bounding boxes. Many types exist, like minimum bounding rectangles (MBR) that try to find the smallest possible rectangle that contains the full shape. We are not doing that here! We're also looking for rectangles just small large enough to fit our shapes, but their edges should be aligned with our x- and y-axes. This is called an axis-aligned bounding box (AABB) and we will be using it for our simulations.

WIP MATHS

Sorting by edges

For efficiency purposes, having a real physical line scan across the scene is not the right way to go. Instead, we take the bounding boxes we found previously and sort the edges in one specific direction. When sweeping in the x direction, we take all vertical edges - each object has exactly two - and sort them from left to right, or negative to positive x. Then we just need to iterate over this list and add/remove objects to/from the tracking list accordingly.

Sweeping

As mentioned before, if at any point the tracking list contains multiple objects, they might be overlapping. For a true overlap, this 'being in the list at the same time' will hold for both directions. So we only mark two objects as collision candidates if they appear together in the tracking list for both the x and y directions.

6.1.2 Binary Space Partitioning

While its name sounds horrendous, binary space partitioning (BSP) is actually a very simple method proposed to prune down the amount of collision candidates. You recurse down a tree, starting from your entire scene of objects. At each node, the current space is cut in two. Think of it like this: you're playing hide-and-seek with your friend in a huge mansion. Since he wants to give you a fighting chance to find him, he allows you to continuously ask binary questions - only two possible answers - such as 'Are you upstairs or downstairs?' and 'Are you in the east or west wing?'. With each question you cut off approximately half of the remaining searching area, until it's become so small you can search on your own. This is the fundamental principle behind BSP. Every node narrows down our scene, until the remaining area covers only a handful of objects. Obviously, any object can only be colliding with an object in the same area partition. It thus becomes a trade-off between recursing deeper to prune more and recursing less to save on computations.

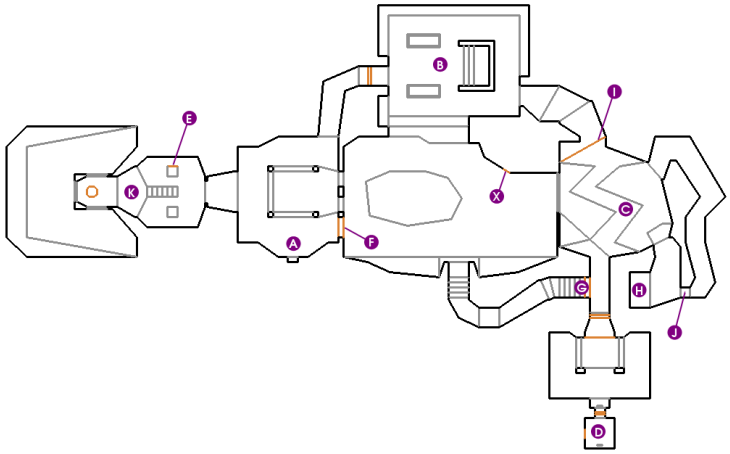


Figure 6.2: First level of Doom viewed top-down

This method is good in theory, but extremely difficult to implement in practice, mainly for the above-mentioned trade-off reason. It is more often used in pre-computed environments like the original Doom's levels (see fig. 6.2), to know which areas to compute and which to set inactive.[11]

6.1.3 Quadtrees

WIP

6.1.4 Octrees

WIP

6.2 Narrow-phase algorithms

After a set of collision candidate pairs has been generated in the broad-phase search, it gets passed on to any of our narrow-phase algorithms. They will determine, with required 100% accuracy, which of these are actually overlapping. This finalized list can be used to find just how bad the overlap is and what the normal/tangential vectors are, for use in dynamic solving.

6.2.1 Separating Axis Theorem

Theorem

The separating axis theorem (SAT) is a theorem in geometry that gives us a way to determine whether two convex shapes are penetrating or not.

Theorem 6.2.1 (Separating Axis Theorem). Two convex shapes A and B do not overlap if and only if there exists a separating axis.

Proof. The proof of this theorem lies far beyond the scope of this introductory course. It is however an extremely intuitive statement. Therefore we will accept this proof by intuition. □

Let's clarify a bit. What are convex shapes and what are separating axes?

A convex shape is a shape for which there exists no straight line that crosses its edges more than twice. The opposite is called a concave shape. The difference is visualized in fig. 6.3.

(a) A convex shape (b) A concave shape

Figure 6.3: Difference between convex and concave shapes [2]

A separating axis is any axis where, when the two shapes are projected onto it, their projections do not overlap. The theorem basically states that if such a separating axis exists, the shapes do not overlap. It also works bidirectionally: if the shapes do not overlap, you are guaranteed to find an axis like that. Or you could invert it and say that if you cannot find a separating axis, the shapes must be overlapping.

Finding separating axes

At this point you might be wondering just which axes we will be checking to determine collision. It would be foolish to just pick N random axes at equidistant angles and hope for the best. In actuality, you only need to check the axes that are normals to the shapes' edges. That means that for two rectangles, you only need to try a maximum of 8 axes - a normal on each side. Since SAT is a falsification theorem, once we find one separating axis we can immediately return

not-overlapping and quit the search. It is only when we've gone through every possible axis with no luck that we can conclude they must be overlapping.

In general, if you're checking for overlap between an n-polygon and an m-polygon, you need to check $n + m$ axes for possible separation. But what if you have a circle? You could argue those are polygons with an infinite amount of sides, but we clearly cannot check infinite axes. Instead, we check the axis formed by connecting the center of the circle to the closest vertex - that is the closest corner of the other shape. If you have two circles, you evidently just need to check the axis connecting both their centers.

Mathematical details: projection

We describe any axis using the angle θ it forms with the x-axis. If $\theta = \frac{\pi}{2}$ we would thus have a vertical line. To project any point (x, y) onto this axis, we take the vector given by (x, y), take the dot product with the unit vector $(\cos\theta, \sin\theta)$ and create a vector in the unit direction with the dot product result as its magnitude. So the projection is given by

$$[x', y'] = [(x\cos\theta + y\sin\theta)\cos\theta, (x\cos\theta + y\sin\theta)\sin\theta] .$$

This equation will be used to project single points onto axes. These could be the vertices of polygons or the centers of circles. In both cases, a simple extension like combining all vertex projections into a line or extending a radius length outwards gives us the shadow of the entire shape on the axis.

Mathematical details: separating axes

We already covered which specific separating axes we needed to check for collisions. In most cases we will end up with a vector (x, y), like the one connecting two circle centers. To get our angle θ from this to define the possible separating axis, we obviously just calculate $\theta = \arctan\frac{y}{x}$, making sure to split off the case where x nears zero, to prevent division by zero.

In the case of a polygon, those normal vector axes are easily found by just getting the edge vector, calculating its θ and adding (or subtracting) $\frac{\pi}{2}$.

The case of concave shapes

The Separating Axis Theorem, as defined before, does not work for concave shapes. This is intuitive, because a concave shape with a little cutout and another shape poking into that cutout would be recognized as a collision by SAT, while it is not one. To fix this, we can divide any concave shape into convex sub-shapes (see fig. 6.4). We then check for proper SAT collision between every pair of A's sub-shapes and B's sub-shapes. If none of the sub-shapes of A overlap with any of the sub-shapes of B, there is no collision.

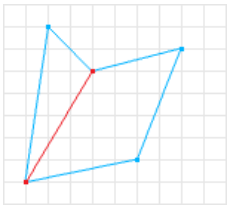


Figure 6.4: A concave shape decomposed into convex sub-shapes

6.3 Contact manifold determination

The final step our collision engine should perform is creating a contact manifold for each colliding pair of objects. The usage of the word "manifold" here will most likely upset mathematicians, since all this is is a concise object carrying all necessary data about the contact, to give to our dynamic engine. So we need to specify the two objects, the normal vector, the tangent vector, and the penetration depth. The last three are used to properly implement the contact and friction constraints, where we need the penetration depth for Baumgarte stabilization.

But that's not all: as we mentioned before, some collisions might require multiple contact points to be taken into account. Like two rectangular objects placed on top of each other: if you use only one contact point and only do constraint solving for that one, nothing is preventing the solver from rotating the rectangles around that point, into each other. So in those cases multiple contact points and equivalently multiple contact/friction constraints per collision can help.

And finally there is the exact location of the contact. This should be a point that lies on both objects. We need its location, because the relative vectors $\vec{q}_i$ that point from each object's center to this point are used for calculating the point's velocity in both bodies - specifically the rotational part $\vec{v}_{rot} = \vec{\omega} \times \vec{q}_i$.

6.3.1 Defining the normal and tangent vectors

For the normal vector $\vec{n}$, which is the direction in which the objects are penetrating each other, you might assume we just draw a vector from one body's center to the other body's center. This might work for simple objects like squares, but in most cases the centers we choose arbitrarily don't have anything to do with penetrations.

Luckily SAT comes to our aid once again. Since we know there is a collision, we can conclude that during narrow-phase, we did not find any separating axes. But we can look at which of our axes had the smallest overlap between the body projections. That axis will be the normal vector direction we're looking for. This is intuitively shown in fig. 6.5.

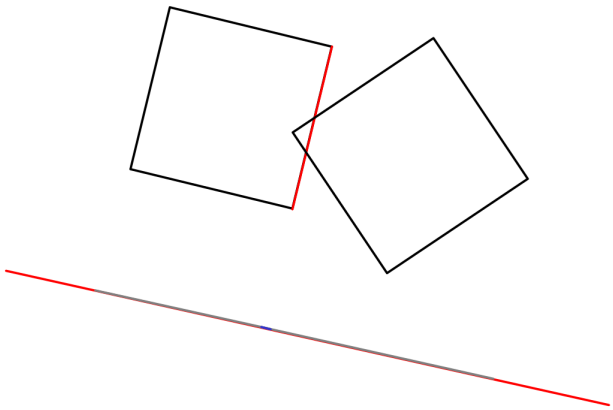


Figure 6.5: Two penetrating black squares and the red axis (normal of red square edge) that minimizes the blue overlap between the gray projections of the squares

The tangent vector $\vec{t}$ is just the perpendicular to $\vec{n}$, which is very easy to calculate.

6.3.2 Defining the contact point(s) and relative vectors

Next up is determining the contact point(s) for this collision. Remember that each contact point will result in a separate contact and friction constraint, so it can build up quickly.

We have to introduce some nomenclature first. We found $\vec{n}$ by figuring out which axis had the least projection overlap. This axis always corresponds to an edge of a polygon. In fig. 6.6 it belongs to the leftmost square. We will call this square the reference object, and the other the incident object. Furthermore, the red edge here is called the reference edge. The incident edge (blue) is the edge for which the dot product of its outward normal with the reference edge's outward normal is smallest - possibly most negative. In other words, this incident edge is the edge most opposite or anti-parallel to the reference edge.

The incident edge has two vertices, shown as green circles here. These vertices are the only possible contact points for our determination. We can clearly tell that the top vertex doesn't qualify as a contact point, but let's determine that procedurally. For each of the green vertices we check the following:

- The reference edge also has two vertices. Now create a 3D plane through each of these vertices, with the normals pointing towards the opposing vertex. In 2D you would see those yellow axes. We now eliminate any green point that is not "inside" both of these planes - the "inside" of a plane is the side to which its normal points. In our example that means eliminating any green point not between the two yellow axes.
- If a green point happens to be eliminated, like our top one, we replace its candidacy by a pink point which is just the intersection between the incident edge and the plane whose "inner-ness" we did not respect - kinda like snapping it back in. This step makes sure any contact points we determine are within the logical boundaries of the reference edge's horizontal.

- The final check is to eliminate any of the remaining points that are inside of the reference edge. The reference edge (red line) obviously also determines a 3D plane, with the normal outward the shape. A point inside of this plane - side outward normal is pointing to - is outside of the reference object's shape. That can clearly not be a contact point, those must be in both shapes! So we only keep the points that are "outside" the reference plane.
- Since we started with 2 possible points - the vertices of the incident edge - the options are that we're left with 0, 1, or 2 contact points. In fig. 6.6 there would be only one. For a collision between the edges of two polygons (like two stacked squares), there would be two. If you don't find any contact points, you must have screwed up somewhere in the earlier steps like the SAT collision determination!

This is an algorithm very similar to the Sutherland–Hodgman algorithm, used to clip a polygon with another polygon.[15]

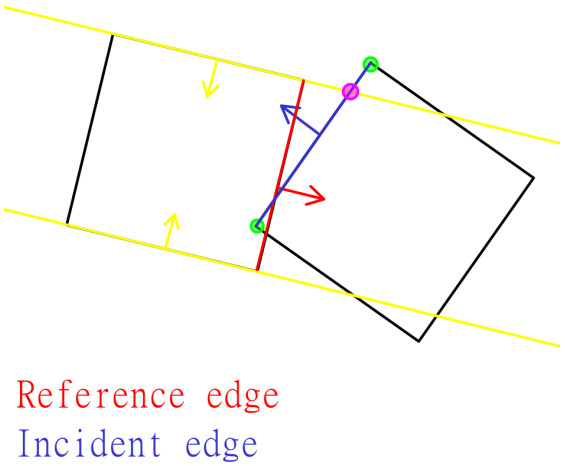


Figure 6.6: Two penetrating squares, the red reference edge, blue incident edge, yellow planes, and corresponding outward normals

Once you have your contact points, you can use them to find the relative vectors pointing from each body's center to them - for rotation in relative velocity.

6.3.3 Defining the penetration depth

Finally there's the penetration depth. Because of its use in Baumgarte stabilization for our contact constraints, each contact point has a different penetration depth to calculate. The reasoning is simple, though. The penetration depth is the perpendicular distance from the contact point to the reference edge. In fig. 6.6 that'd be the perpendicular distance between the bottom green point and the red reference edge!

6.4 Preventing tunneling

Something that comes up often when discussing collisions in simulations or games, is the effect of tunneling. Imagine you have a thin wall and an object heading straight for it at a rapid pace.

It's possible that given its high velocity, a position time-step could make it go from 'right in front of wall' to 'right behind wall' - see fig. 6.7. This phase-through is what we need to prevent.

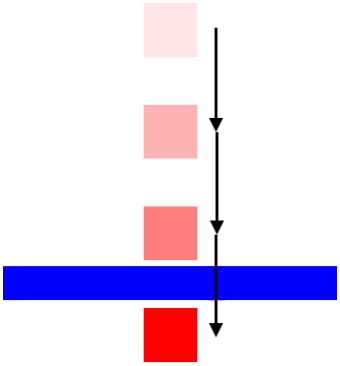


Figure 6.7: A fast square tunneling through a thin wall
[8]

The effect can be caused by a combination of high speeds, large time-steps Δt, and thin object collisions. Solving it, however, is not as simple as lowering Δt and hoping a more time-detailed simulation will cause collision preventing tunneling.

The real solution to this conundrum will be Continuous Collision Detection (CCD). As the name implies, we will use continuous approximations to zoom into time-steps and figure out which objects will cross each other. The name is also a bit of a lie though, since doing something continuous in programming is by definition impossible - that'd require infinitesimally small time-steps...

CCD is a trade-off: we're having to come up with approximation methods to detect collisions before they happen discretely, and so we're bound to make mistakes. Simulations can never be 100% correct. Therefore we'll only briefly cover the two main ways CCD can work, as well as their downsides. When implementing the engine, we'll focus on the main components first, before optionally extending it with a CCD algorithm.

6.4.1 Sweep-based CCD

The first technique is called sweep-based CCD. As the name implies, we take an object and its velocity, then sweep out its predicted path for this time-step and check if any other objects are on that path. If there are, we estimate when that collision would happen in time and split the current time-step into sub-steps accordingly, making sure the collision actually occurs and gets detected properly.[10]

This sounds amazing on paper, but has a lot of downsides:

- It requires its own (preferably) fast collision algorithm, between swept paths and other objects.
- It demands a ton of computations, especially if there are a lot of objects present in your simulation. The computational speed can also be tanked by faster objects, as they sweep out larger areas in the same time-step.

- It is unable to prevent tunneling for rotational movements, due to the extra simulating that would require.

6.4.2 Speculative CCD

The second technique is called speculative CCD. For any two objects we can calculate their relative velocity and project it onto the normal between them. This was used before for contacts, when objects physically collided with each other - we then forced this velocity to be zero or positive. If objects are not yet penetrating, but they might cross over each other, we can introduce a speculative contact constraint. It enforces

$$\vec{n} \cdot (\vec{v}_1 - \vec{v}_2) \geqslant -\frac{\Delta r}{\Delta t}$$

meaning the relative velocity *can* be negative, but only up to the specified amount. The amount in question is just the speed you'd need to bridge the gap (Δr) between the two objects in one time-step (Δt). So if two objects are 5 units apart and a time-step is 1 second, the speculative constraint just says these objects cannot have a normal relative velocity higher than 5 units per second (in the negative direction).

But do we need to add these speculative constraints for all pairs of objects? Obviously not! If in the current frame the two objects have too little relative velocity to cross the gap in a single time-step, they probably shouldn't be considered for a speculative constraint. I say 'probably' since the velocities can change in the solver, so we need a bit of margin. Let's say that if $\vec{n} \cdot (\vec{v}_1 - \vec{v}_2) \leqslant -\frac{\Delta r}{\Delta t} + m$, with m an error margin, we should consider adding a speculative constraint.

There's also the fact that these non-colliding pairs we want speculative contacts for, might not even be selected in the broad-phase collision detection. So we also change the broad-phase step: extend an objects AABB to cover both its current position and its predicted position in the next frame, using the current velocity. This will make the new AABB overlap anything that the object might tunnel through, and broad-phase detection will pass it on to narrow-phase. Inside of narrow-phase, we still start by using SAT for the final elimination, but instead of throwing all other candidates away immediately, we check the condition from before to maybe add that speculative constraint.[9]

Chapter 7

Designing an engine

7.1 Full simulation layout

Take a breather, because at this point we've covered basically everything you need for your first genuine 2D physics engine. To recap, the entire structural diagram is visualized neatly on fig. 7.1.

During the process of explaining each component in the previous chapters, we didn't always go too in-depth on practical or computational implementations. That's what this (disproportionately long) chapter is for! Because of its length, let's go over the exact remaining steps for engine implementation.

1. We decide a programming language to implement the engine in.
2. We design before we program, using class diagrams and layouts.
3. We create all necessary files, empty implementations, and specifications.
4. We implement from root to most dependent, building up the engine.
5. We write a simple renderer.
6. We perform a multitude of tests.
7. We recreate the first level of Angry Birds with the engine.

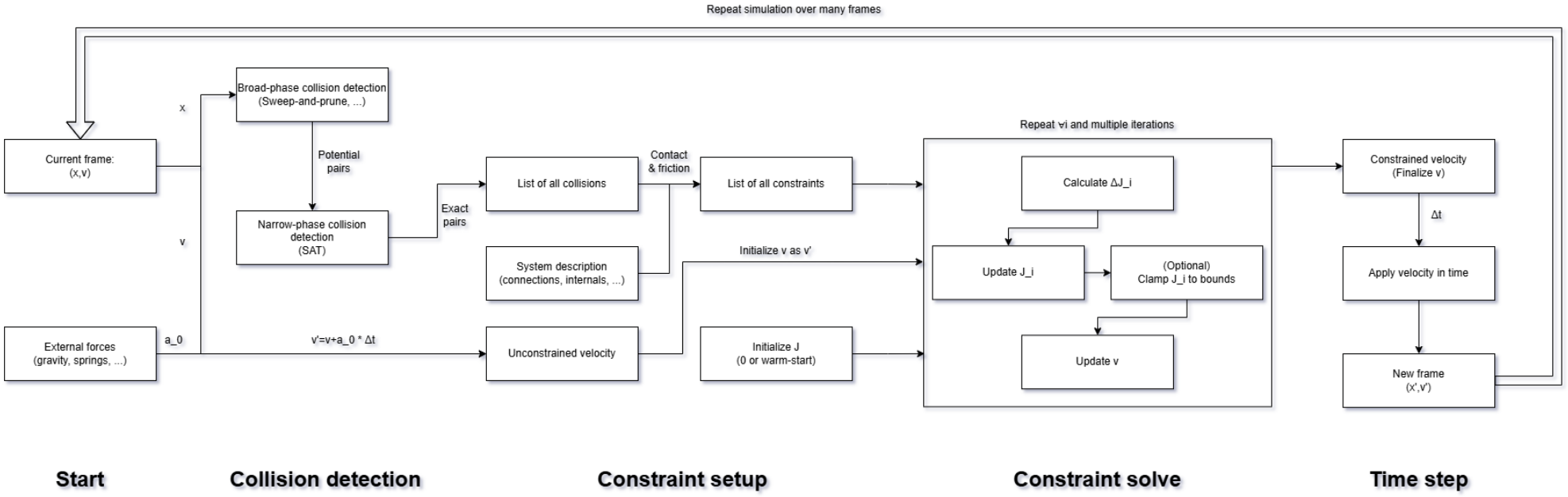


Figure 7.1: Full schematic representation of the 2D physics engine

7.2 Choice of programming language

This 2D physics engine can hypothetically speaking be implemented in any and all programming languages. However, given the nature of the system, some will make this job easier than others.

Physics engines are all about objects and how they interact with each other. What I just said is also a description of object-oriented programming languages (OOPs). That's why the language we choose should probably be one of those. We can then design the class diagram first, with all necessary classes, methods, associations, inheritances, etc.

That still leaves us with quite a few options, but the main ones to consider are Python, C, C++, and Java. Python would be a beginner-friendly choice, but we won't be using it because it technically is not just an object-oriented programming language. Python also doesn't benefit from compiler speedups or proper type-checking to prevent fatal errors. Since I enjoy the outside world, we also won't be using good old C. So the decision is between C++ and Java.

A popular poll would probably give C++ the lead, since it is better suited for mathematical and scientific programming. We however will be choosing Java, partly because it's more user-friendly and partly because I followed a Java course in my undergraduate degree.

7.3 Class diagram design

7.4 Implementing classes

7.4.1 Bodies

7.4.2 Force generators

7.4.3 Constraints

7.4.4 World

7.4.5 Physics loop

7.5 Pixel rendering

7.5.1 Filling polygons using scanline rasterization

7.5.2 Sub-pixel antialiasing

7.6 The user experience

7.6.1 Tick listeners

7.6.2 Examples

Chapter 8

Extra: Extension to 3D

Bibliography

[1] Joachim Baumgarte. Stabilization of constraints and integrals of motion in dynamical systems. *Computer methods in applied mechanics and engineering*, 1(1):1–16, 1972.

[2] William Bittle. SAT (Separating Axis Theorem), January 2010.

[3] Erin Catto. Physics for game programmers: Understanding constraints, November 2016. GDC 2014 talk, Internet Archive.

[4] Erin Catto. Solver2D, February 2024.

[5] Erin Catto and Crystal Dynamics. *Iterative Dynamics with Temporal Coherence*. June 2005.

[6] Herbert De Gersem and Eef Temmerman. *Klassieke Mechanica*. Acco, 2024.

[7] Carl Friedrich Gauß. Über ein neues allgemeines grundgesetz der mechanik. 1829.

[8] Kenton Hamaluik. Swept AABB collision detection using the Minkowski difference.

[9] Unity Technologies. Unity - Manual: Speculative CCD.

[10] Unity Technologies. Unity - Manual: sweep-based CCD.

[11] Newcastle University (unknown author). Physics - Broad phase and Narrow phase. *Game Engineering Masters Degree*.

[12] Wikipedia. Gauss's principle of least constraint — Wikipedia, the free encyclopedia. `http://en.wikipedia.org/w/index.php?title=Gauss's%20principle%20of%20least%20constraint&oldid=1313487261`, 2026. [Online; accessed 07-January-2026].

[13] Wikipedia. Lagrange multiplier — Wikipedia, the free encyclopedia. `http://en.wikipedia.org/w/index.php?title=Lagrange%20multiplier&oldid=1320521305`, 2026. [Online; accessed 07-January-2026].

[14] Wikipedia contributors. Karush–kuhn–tucker conditions — Wikipedia, the free encyclopedia, 2025. [Online; accessed 15-January-2026].

[15] Wikipedia contributors. Sutherland–hodgman algorithm — Wikipedia, the free encyclopedia, 2025. [Online; accessed 22-March-2026].

Acknowledgment of GenAI usage

Generative AI chatbots including Google's Gemini, Anthropic's Claude, and OpenAI's ChatGPT were used several times during the writing of this text. They were utilized to discover external papers, courses, and other material which has been properly cited in bibliography. No statements made by AI were taken as fact without external verification.

KU Leuven Kulak
Faculty of Science & Technology
Etienne Sabbelaan 53, 8500 Kortrijk
Tel. +32 12 34 56 78
casper.vermeeren@student.kuleuven.be